Projektuppgift

Introduktion till webbutveckling med HTML,CSS och JavaScript, DT224G

Projektuppgift
LekZonen

Victor Heras



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Victor Heras, visa2601@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

Hundägare idag som vill att deras hundar ska få möjlighet att umgås och leka med andra hundar saknar en bra plats att möjliggöra detta. LekZonen är en organisation som ger en webbplats där hundägare kan hitta träffar i sin stad eller skapa en träff själv för att kunna ge sin hund en möjlighet att leka med andra.

Webbplatsen består av tre sidor: start, träffar och kontakt, där träffar sidan visar befintliga träffar eller möjligheten att skapa en träff själv. Webbplatsen planerades med wireframes och mockups med tanken att skapa miljön främst för mobila användare och sedan utvecklad i Visual Studio Code med HTML, CSS och JavaScript.

Länk till webbplatsen finns här: https://lekzonen.vercel.app/

Innehållsförteckning

Sammanfattning ..... **iii**

1 Introduktion / Inledning **1**
1.1 Problemmotivering och problemformulering 1
1.2 Avgränsningar 1

2 Teori / Bakgrundsmaterial **2**
2.1 HTML, CSS och Javascript 2
2.2 Responsiv webbdesign och mobile first 2
2.3 DOM och eventhantering 2
2.4 LocalStorage och JSON 3

3 Metod **4**

4 Konstruktion **6**
4.1 Struktur 6
4.2 Responsiv layout och mobilmeny 6
4.3 Skapa träff 7
4.4 Filtrering för stad 9
4.5 Kontaktformulär 10
4.6 Publicering 11

5 Resultat **12**

6 Slutsatser / Analys / Diskussion **14**

Källförteckning **16**

1 Introduktion / Inledning

LekZonen är en organisation som vill göra det enklare för hundägare att hitta träffar där deras hund/hundar kan få möjligheten att leka med andra hundar. Där målgruppen är för personer med hund som i sin egna familjekrets eller vänskapskrets inte har ett alternativ för sin hund att socialisera sig med andra hundar. Det allra flesta idag har mobilvana där hundägare som ofta är ute med sina hundar med störst sannolikhet har med sig sin mobiltelefon, vilket gör att tjänsten LekZonen erbjuder kommer främst behöva fungera på mindre skärmar.

Webbplatsens syfte erbjuder därför en inbjudande och enkelhet till hundägare för att gå från hemsidan till att träffa andra hundägare och hundar.

1.1 Problemmotivering och problemformulering

Hundägare idag som vill få ge sina hundar möjlighet till socialisering med andra är oftast hänvisade/går genom en uppsjö av spridda grupper på olika socialmedier exempelvis Facebook. Dessa grupper brukar till störst del också vara anpassad för en specifik ras vilket kan bli problem för personer med någon ovanligare ras. Samma gäller även hundens storlek där exempelvis små och stora hundar kan kännas obekväm, olämplig, fel i vissa kombinationer. Det saknas därför en simpel och lätt plats att hitta träffar.

LekZonen behöver därför uppfylla:

- Hitta träffar: Hundägaren skall enkelt kunna se befintliga träffar.
- Skapa träffar: Hundägaren skall kunna skapa ett eget event.
- Fungera på mobil: Webbplatsen skall vara enkel att navigera på sin mobiltelefon.

1.2 Avgränsningar

Webbplatsen är en prototyp utan server. Det innebär att ingen delning mellan enheter är för stunden fungerande.

2 Teori / Bakgrundsmaterial

2.1 HTML, CSS och Javascript

I grunden är en webbplats uppbyggd på tre delar som har olika roller. HTML är delen som beskriver innehållet på sidan, så som rubriker, stycken, bilder, formulär etc.[1]. CSS är delen som styr utseendet hos delarna från HTML exempelvis som färg, typsnitt, storlek, placering etc. [2]. JavaScript är ett programmeringsspråk som gör det möjligt att reagera på användares handlingar, exempelvis att läsa av ifall ett användarnamn med lösenord är korrekt eller ej[3].

2.2 Responsiv webbdesign och mobile first

Responsiv webbdesign innebär att webbplatsen kan anpassa sin utseende efter användarens skärmstorlek. Detta för att en sida skall kunna fungera bra på mobiltelefoner, surfplattor samt dator. Ett vanligt sätt som även denna webbplats använder sig av är så kallad mobile first vilket innebär att man först utgår från designen för en mobilanvändare, där sedan med hjälp av något som heter media queries ändrar utseendet för andra skärmstorlekar som datorskärm[4].

2.3 DOM och eventhantering

När en webbplats laddas in bygger webbläsaren sidans HTML som ett träd av olika objekt som kallas DOM (Document Object Model)[5]. Med hjälp av JavaScript kan man hämta element från ur HTML trädet och exempelvis ändra dess innehåll och klasser och läsa händelser som klick eller om musen håll över. En så kalla ”eventlistener” kopplas med ett element med hjälp av addEventListener och körs när den specifika typen av händelse sker hos elementet.

2.4 LocalStorage och JSON

En del av webbläsarens webblagring är localStorage vilket gör det möjligt att spara data på användarens egna enhet. Denna data sparas även om sidan stängs ner eller laddas om. LocalStorage kan bara spara/lagra data som strings och därför används JSON. JSON.stringify gör upp arrays och objects till text där JSON.parse gör texten tillbaka till array eller object[6].

3 Metod

Det första som gjordes i arbetet var att komma på, skriva ner och formulera iden samt dess behov som nämndes i kapitel 1.1. Efter detta skapades en wirefram av webbplatsen alla sidor för både mobil och dator, där sedan utefter wireframesen gjordes mockups. Wireframesen och mockupsen gjordes allt i figma. Alla tre sidor: index, meets, contact på webbplatsen hade wireframe samt mockups för båda mobil samt dator. Anledningen till detta var att underlätta nästkommande del i arbetet. Om wireframes gjordes för alla tre sidor på både mobil och dator underlättades det att göra mockups, som i sin tur underlättade att när väl kodning påbörjades då alla beslut var tagna och inget nytt behövde kommas på. Detta var iden från början för att underlätta hela processen för arbetet.

Riktningen för designen utgick från många personliga val, där bland annat lila användes efter favoritfärg, typsnittet Baloo 2 efter djungelboken men även dess rundhet i typsnittet som även resterande delar av webbplatsen använder sig av exempelvis träff puffarna.

Webbplatsen kommer sedan skapas med HTML, CSS och Javascript. Eftersom målgruppen är ute och har mest tillgång till sina telefoner kommer webbplatsens CSS först designas utefter detta och använda sig av CSS Flexbox då layouten på telefoner görs att allt oftast ordnas i en riktning.

Webbplatsen kommer inte använda sig av en server eller databas och istället använda sig av localStorage. LocalStorage är ett enkelt sätt att spara data för träffar samt meddelanden från kontaktformuläret.

Formulären för både skapa träff samt kontakt kommer att kontrolleras med hjälp av JavaScipt och inte använda HTML attributet ”required”[7]. Detta gör exempelvis att båda villkor för fält som måste fyllas i och eventuella felmeddelanden kan hanteras av endast JavaScipt.

All kod i projektet kommer att versionshanteras med hjälp av Git. Detta för att kunna följa ändringar under utvecklingen. Det kommer även publiceras till Vercel via Github som kommer bygga hemsidan och sätta webbplatsen i drift.

4 Konstruktion

4.1 Struktur

LekZonen består av index.html, meets.html och contact.html, som alla delar samma CSS fil (style.css) där sedan varje HTML fil har en egen JavaScript fil med samma namn, exempelvis index.js. De tre sidorna delar samma header med logga, namn på hemsidan samt navigation och delar även samma footer. Detta gör att samma HTML kod kan upprepas för headern och footer på alla tre HTML sidor.

4.2 Responsiv layout och mobilmeny

CSS skrevs enligt mobile first vilket innebär att grundlayouten är anpassad för mindre skärmar som telefoner där en media query används sedan för att anpassa hemsidan för större skärmar.

```
/*----@MEDIA för desktop----*/

@media (min-width: 768px){
```

Figur 1 – Media Query där CSS för större skärmar finns

I headern på mobil är navigationen dold bakom en meny knapp, där när meny knappen klickas på öppnas och stängs navigations menyn. Detta görs genom att i JavaScript koden läggs en klass till/tas bort i HTML elementet där sedan CSS filen visar eller inte beroende på klassen. JavaScript koden hämtar knapp-elementet och menyn ur DOM och genom en eventlistnare[8] lyssnar och väntar på ett klick på meny knappen. Vid varje klick växlas klassen på meny elementet med klassen open där CSS har olika regler för klassen ”menu” och ”menu.open”.

```
const menubutton = document.querySelector(".menu-button")
const menu = document.querySelector(".menu")

//Öppnar och stänger menyn i header på mobil
menubutton.addEventListener("click", function(event){
    event.preventDefault();
    menu.classList.toggle("open")
})
```

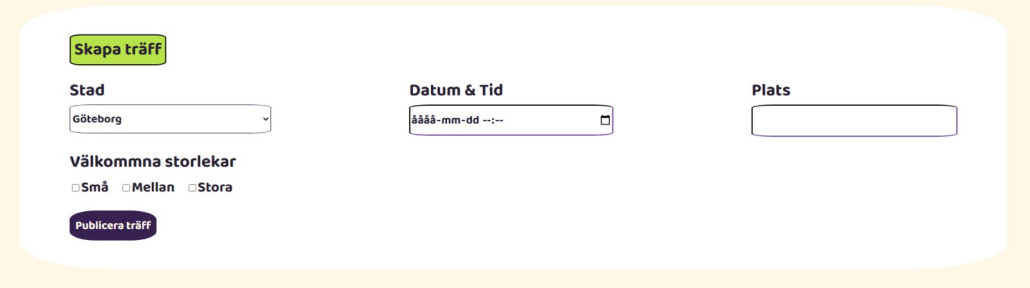
Figur 2 – JS kod för att öppna och stänga mobilmenyn

```
.menu{
    display: none;
}
.menu.open{
    display: flex;
```

Figur 3 – CSS för när menyn är öppen/stängd

4.3 Skapa träff

I meets.html finns en skapa träff knapp som öppnar upp ett formulär man kan fylla i för att själv skapa ett event. Formuläret innehåller en rullmeny för stad, ett fält för datum och tid, ett fält för plats samt tre stycken rutor för storlek på hunden.



Figur 4 – Skapa träff formuläret på sidan

I JavaScript koden finns en funktion som när den kallas kollar genom att stad, datum och tid, plats samt minst en av storleksrutorna är ikryssade. Funktionen tömmer först eventuella tidigare felmeddelanden och kontrollerar sedan att de fält som krävs är ifyllda. Om allt är korrekt kommer funktionen returnera true, annars kommer funktionen skriva ut ett felmeddelande till DOM på de fält som ej är ifyllda och funktionen kommer returnera false.

```
function publishCheck(){

    //Tömmer errormeddelanden
    missedFields.innerHTML = ""

    //variabel som eventuellt om en if går genom sätts till 1 vilket inte kommer att ge retun = true
    let correct = 0;

    if(selectTimeDate.value === ""){
        missedFields.innerHTML += "<p>Vänlig välj datum och tid</p>";
        correct = 1;
    }
    if(selectPlace.value === ""){
        missedFields.innerHTML += "<p>Vänlig välj plats för träffen</p>";
        correct = 1;
    }
    if(checkboxS.checked === false && checkboxM.checked === false && checkboxL.checked === false){
        missedFields.innerHTML += "<p>Vänlig fyll i välkomna storlekar</p>";
        correct = 1;
    }
    //return true om correct är lika med 0
    return correct === 0;
```

Figur 5 – JS kod för att validera inmatning av fält skapa träff formuläret

När publishCheck returnerar true kommer en funktion createcardMeet kallas som läser av fälten från formuläret och sparar allt i ett objekt som sedan sparas i en array som heter meets. Meets arrayen görs om till JSON i funktionen store-Mees för att spara i localStorage och i funktionen loadMeets kan alla data från localStorage göras om från JSON tillbaka till en array.

```
function createcardMeet(){
    //Hämtar data från create-meet form
    const eventMeet = {
        city: selectCity.value,
        timedate: selectTimeDate.value,
        place: selectPlace.value,
        small: checkboxS.checked,
        medium: checkboxM.checked,
        large: checkboxL.checked,
    };

    //Lägger eventMeet i meets
    meets.push(eventMeet);

    //Sparar meets
    storeMeets();
    //returnerar eventMeet för användning i annan function
    return eventMeet;
}
```

Figur 6 – Funktion som läser av fälten från formuläret av sparar i localStorage

```
function storeMeets(){
    localStorage.setItem("meets", JSON.stringify(meets));
}
//Läser in meets från localstorage
function loadMeets(){
    //Hämtar eventuell sparad meets
    const savedMeets = localStorage.getItem("meets")

    //uppdaterar meets[]
    if (savedMeets){
        meets = JSON.parse(savedMeets);
    }
}
```

Figur 7 – Funktioner som spara och hämtar från localStorage

4.4 Filtrering för stad

Varje träffkort får stadens namn som klass och genom detta byggs hela filtreringen på. När en av sorterings knapparna klickas får den knappen ett klassvärde ”pressed”. När funktionen chosenCity kallas läser den av vilket av sorterings knapparna som innehåller klassen ”pressed”. Om Göteborg knappen är i klickad kommer funktionen returnera Göteborg, Stockholm returnera Stockholm och Malmö returnerar Malmö. Annars returneras ”alla”. Funktionen sortMeets går genom alla träffkort och tar bort klassen ”show” på alla först för att sedan gå genom alla träffkort med klassen från det chosenCity returnerade och lägger till värdet ”show”. Sedan i CSS finns design för värdet för show medan de utan visar ingenting.

```
function chosenCity(){
    if(cGBG.classList.contains("pressed")){
        return "Göteborg";
    };
    if(cSthlm.classList.contains("pressed")){
        return "Stockholm";
    };
    if(cMal.classList.contains("pressed")){
        return "Malmö";
    };
    return "alla";
}

//function som sorterar vilka meet puffar som skall visas
function sortMeets(city){
    const cardMeet = document.querySelectorAll(".meetups article")

    for (let i=0; i<cardMeet.length; i++){
        //Tar bort eventuellt show från class
        cardMeet[i].classList.remove("show");

        //om city = "alla" eller article class i .meetups innehåller samma stad läggs "show" till i klassen
        if(city === "alla" || cardMeet[i].classList.contains(city)){
            cardMeet[i].classList.add("show");
        }
    }
}
```

Figur 8 – Funktioner som läser av sorteringsknapp och sorterar

4.5 Kontaktformulär

Kontakt formuläret i contact.html har tre fält: namn, e-post och meddelande. Valideringen här följer samma princip som valideringen i skapa träff använder sig av där tomma fält ger felmeddelande och när alla är ifyllda visas ett tack-meddelande och alla uppgifter sparas i localStorage.

```
contactButton.addEventListener("click", function(event){
    event.preventDefault();

    if(checkContact()){
        contactmessage.innerHTML = "<p class='tnxmsg'>Vi besvarar din fråga så fort vi kan :)</p>"
        const contactQ = {
            name: cName.value,
            email: cEmail.value,
            Message: cMessage.value,
        }

        questions.push(contactQ);

        localStorage.setItem("questions", JSON.stringify(questions));

    }
})
```

Figur 9 – Eventlistener som kollar när kontakt knappen klickas

4.6 Publicering

Koden har versionshanterats med Git, där ändringar har sparats löpande som commit med meddelanden. Repositorit publicerades på Github som sedan kopplades till Vercel som sätter webbplatsen i drift.

5 Resultat

Webbplatsen består av tre sammanlänkade sidor som delar header och footer men med olika main delar. En navigation finns på alla sidor där på telefon visas den som en meny knapp som öppnas och stängs.

Startsidan index.html presenterar organisationen med bilder hundar som lekar och umgås, en kort text om vad organisationen erbjuder och knappar som leder till träff sidan.

Träffsidan meets.html innehåller filterknappar för stad, en knapp som öppnar upp ett formulär för att skapa en träff själv samt de träffar som är skapade. De skapade träffarna visas som ett litet kort med information om vilket stad, välkomna storlekar, tid och plats. Sidan visar endast skapade träffar så om inga träffar finns är det annars tomt, träffarna finns även kvar när sidan laddas om.

Kontaktsidan contact.html har ett formulär med namn, e-post och meddelande för att ställa frågor samt direkt kontakt organisationen själv.

Åter koppling till rubrik 1.1 i rapporten:

Behov	**Lösning i webbplatsen**
Hitta träffar	Träffsidan med filter för stad
Skapa träffar	Formulär för att skapa träffar
Fungera på mobil	Mobile first-CSS och menyknapp för mindre skärmar

Figur 10 – Behov och dess lösningar

Test resultat:

Test	**Resultat**
Öppna och stänga mobilmenyn	Menyn öppnas och stängs vid klick ✅
Navigera mellan sidorna	Rätt sida öppnas vid varje länk ✅
Skapa träff med tomma fält	Felmeddelande visas och ingen träff skapas ✅
Skapa träff med ifyllda fält	Ett kort visas på sidan med korrekt information ✅
Filter för stad	Bara träff för vald stad visas ✅
Ladda om träffsidan	När sidan laddas om visas skapade träffar ✅
Skicka tomt kontaktformulär	Felmeddelande för varje fält visas ✅
Skicka ifyllt kontaktformulär	Tackmeddelande visas ✅
W3C CSS validering	Ett valideringsfel ❌
W3C HTML validering	Inga validerings fel ✅

Figur 11- Resultat av tester

6 Slutsatser / Analys / Diskussion

Webbplatsen uppfyller till stor del det behovet som fanns. Besökare kan se träffar och filtrera stad, kan skapa träffar själva och är enkel att använda båda på större som mindre skärmar. Begränsningen som finns är att webbplatsen använder sig av localStorage och sparar träffarna i sin användarens egna webbläsare. Därför ser bara användare just nu bara träffar personen själv har skapat och inte träffar från andra. Det blir då en brist då det inte går att skapa träffar mellan hundägare.

Själva funktionen att skapa träffar är uppfyllt. Formuläret har krav på fält som behöver fyllas i och valideringen förklarar eventuella fält som ej är ifyllda. Valet av rullista för stad gjorde det enklare vid sortering och minskade möjliga felinmatningar. Punkter som hade kunnat förbättras för funktionen är att inte välja datum som passerats, just nu kan man det vilket blir konstigt om man skapar en träff som hålls för ett år tillbaka… Datumet visas också direkt i den form som inmatningen ger ut år-månad-dagTtimme:minut. Exempelvis 2026-11-04T12:15, där en förbättring hade varit att ta bort ”T” mellan datumet och tiden. En annan förbättrings del hade även varit att även sortera efter datum och tid där nästkommande låg först.

Behovet för kontaktformulär är uppfyllt och ger bra återkoppling. Den brist som borde nämnas är att ingen kontroll att det är en e-post som angivits i e-post fältet.

Webbplatsen fungerar bra på mobil och skulle sägas är bäst byggd utefter mobil. Wireframes och mockups gjorde att allt arbete för senare underlättade för att få denna byt att bli som smidig som möjlig vid kodningen.

Den största ändringen som skedde är att ursprungligen fanns det en plan att även kunna sortera för storlek. Man skulle då kunna sortera efter både stad och storlek. När denna delen skulle implementeras kom ingen lösning på tillräckligt fort för att det skulle kännas värt att implementera och därför togs bort. Koden finns fortfarande kvar i HTML filen, CSS samt JavaScript men är bort kommenterad för eventuella användning vid utveckling i framtiden.

Eventuella förbättringar i framtiden är att införa server och databas för att kunna dela träffar mellan användare. Implementation av användarkonton skulle isåfall också göras. Självklart också storleks sorteringen hade också implementerats.

Slutsatser om webbplatsen är att den uppfyllde dom tre behoven som nämndes, valet av göra alla wireframes och mockups gjorde hela processen enklare. Där mobile first gav en bra riktning för webbplatsen. Samtidigt visar det även att localStorage inte är en bra alternativ för denna webbplatsen då det inte är ett alternativ för att kunna dela träffar mellan användare och skulle därför kräva server och databas.

Källförteckning

Här följer exempel på hur en källförteckning kan utformas enligt Vancouver-systemet. Den är automatiserad enligt metoden numrerad lista och korsreferenser, som beskrivs i kapitel 2.4.

[1] Webbutvecklingsprogrammet. Introduktion till HTML. 2026, 19 augusti [citerad 2026.10.31]. Hämtad från: https://www.youtube.com/watch?v=zx1wKb1cehE

[2] Webbutvecklingsprogrammet. Introduktion till CSS. 2026, 11 Juni [citerad 2026.10.31]. Hämtad från: https://www.youtube.com/watch?v=t9H1ma-xsog

[3] Lars Lundin. JavaScript. 2026, 26 Augusti. [citerad 2026.10.31]. Hämtad från: https://www.youtube.com/watch?v=UVc3rxWSjuw

[4] W3schools.com. HTML Responsive Web Design [Internet]. [citerad 2026-10-31]. Hämtad från: https://www.w3schools.com/html/html_responsive.asp

[5] W3schools.com. The HTML DOM [Internet]. [citerad 2026-10-31]. Hämtad från: https://www.w3schools.com/js/js_htmldom.asp

[6] W3schools.com. JavaScript JSON [Internet]. [citerad 2026-10-31]. Hämtad från: https://www.w3schools.com/js/js_json.asp

[7] W3schools.com. HTML <input> required Attribute [Internet]. [citerad 2026-10-31]. Hämtad från: https://www.w3schools.com/tags/att_input_required.asp

[8] W3schools.com. JavaScript EventListener [Internet]. [citerad 2026-10-31]. Hämtad från: https://www.w3schools.com/js/js_htmldom_eventlistener.asp